

UNIVERSIDAD PERUANA CAYETANO HEREDIA

FACULTAD DE CIENCIAS E INGENIERÍA

“ALBERTO CAZORLA TALLERÍ”



TEMA:

Sistema de Recomendación de Películas

CURSO:

Métodos Numéricos y Optimización para Machine Learning

ALUMNOS:

- ANDRES STEVEN RODAS JANAMPA
- MARCELO JOEL RODRIGUEZ CABRERA
- PATRICIA DEL PILAR REBATTA JERI

PROFESORES:

- JOSUE ANGEL MAURICIO SALAZAR
- JEREMY KARSEN HOLGUIN MORI

**Lima - Perú
2025**

Abstract

El fenómeno de la sobreabundancia de información en las plataformas de streaming modernas genera fatiga de decisión en los usuarios, requiriendo sistemas eficientes de filtrado de contenido. El presente proyecto, denominado MOVIE_ML, desarrolla un ecosistema híbrido de recomendación basado en el dataset TMDb para mitigar este problema mediante técnicas de álgebra lineal y optimización numérica. La metodología se estructura en un flujo de procesamiento de datos riguroso que aborda el problema de la dispersión matricial (*sparsity*) mediante la normalización heterogénea de metadatos. Se implementa un motor de filtrado basado en contenido, el cual consolida atributos cualitativos en un espacio semántico (*Tag Soup*) y emplea vectorización TF-IDF con normalización L2 para calcular la Similitud del Coseno, resolviendo eficazmente el problema de arranque en frío (*cold start*). Adicionalmente, para capturar patrones de comportamiento latentes, se diseña un entorno de simulación poblacional con usuarios sintéticos, sobre el cual se ejecuta una arquitectura de optimización mediante Factorización de Matrices y Descenso de Gradiente Estocástico (SGD). La evaluación sistémica del modelo se alinea con estándares industriales (Precisión@K y RMSE), demostrando cómo la intersección entre ingeniería de características y métodos probabilísticos permite construir una herramienta altamente escalable que transforma el gusto cinematográfico en una variable estrictamente cuantificable y geométricamente optimizable.

INTRODUCCIÓN

En la era de la economía digital, las plataformas de streaming enfrentan el fenómeno de la sobreabundancia de información, donde la inmensidad de los catálogos genera en el usuario una fatiga de decisión que degrada la experiencia de consumo. Ante esta necesidad crítica de filtrar contenido de manera eficiente, nace el proyecto MOVIE_ML: un ecosistema de recomendación construido sobre el dataset de TMDb que transforma la búsqueda de contenido en una experiencia de descubrimiento geoméricamente optimizada.

El núcleo técnico del sistema reside en la aplicación de álgebra lineal y métodos numéricos de alto rendimiento. Mediante la vectorización de metadatos cualitativos (reparto, géneros y dirección) en espacios multidimensionales, el sistema emplea el cálculo de la similitud del coseno para determinar la proximidad geométrica entre contenidos, tratando el gusto cinematográfico como una variable cuantificable.

No obstante, el rigor del proyecto trasciende la comparación de atributos al implementar una arquitectura de Factorización de Matrices optimizada mediante el Descenso de Gradiente Estocástico (SGD). Debido a la ausencia de interacciones reales en el conjunto de datos original, se desarrolló un entorno controlado de usuarios sintéticos y calificaciones simuladas bajo distribuciones normales, permitiendo al algoritmo minimizar iterativamente la función de pérdida y capturar factores latentes de preferencia. A través de este enfoque híbrido, el proyecto demuestra cómo la intersección entre la subjetividad humana y la precisión algorítmica permite construir una herramienta que se ajusta matemáticamente a la identidad de cada usuario.

ESTADO DEL ARTE

1. Ingesta y Consolidación

El Problema de la Dispersión en cualquier sistema de recomendación, el número de valoraciones disponibles es significativamente menor al total de valoraciones posibles. Esta brecha genera una matriz usuario-ítem escasa que degrada la precisión predictiva de los algoritmos.

El proyecto MovieLens, desarrollado por GroupLens Research, fue el primero en demostrar este fenómeno a escala real. Al consolidar millones de valoraciones de usuarios reales, documentaron que la dispersión (*sparsity*) supera el 95% de entradas sin valorar y constituye uno de los principales obstáculos para los sistemas colaborativos cuando las matrices son muy escasas, los algoritmos de vecindad más cercana fallan al no encontrar suficientes usuarios similares [1]. A medida que crece el catálogo de ítems, la dimensión de la matriz y su dispersión aumentan exponencialmente, afectando directamente la calidad de recomendación [1].

Para mitigar este problema, el presente proyecto adopta una estrategia de normalización de fuentes heterogéneas, integración matricial con integridad de metadatos y transformación vectorial previa al modelado, siguiendo los principios de preprocesamiento documentados por GroupLens [1].

2. Características TF-IDF y Espacio Vectorial

El Proyecto Genoma Musical de Pandora es uno de los casos más representativos de ingeniería de características aplicada a recomendación basada en contenido. Sus ingenieros clasifican cada canción mediante cientos de "genes musicales" atributos como ritmo, tonalidad, instrumentación y letra que luego son vectorizados para calcular similitudes entre ítems [2]. Este enfoque es estructuralmente análogo al "Tag Soup" implementado en este proyecto: la concatenación de género, director, actores y sinopsis de cada película para construir un espacio vectorial de características.

Se establece que el pesado TF-IDF es el método estándar para transformar atributos cualitativos en vectores numéricos. El término TF mide la frecuencia local de un término en el ítem; el IDF penaliza términos muy comunes que aportan poco poder discriminativo:

$$TF - IDF(t, d) = TF(t, d) \times \log \log \left(\frac{N}{df(t)} \right)$$

La normalización L2 escala cada vector al hiperplano unitario, haciendo que la similitud del coseno sea invariante a la longitud del documento:

$$\hat{v} = \frac{v}{\|v\|_2}$$

Una aportación clave de este paradigma documentada también por Pandora es resolver el problema de *cold start* al depender de atributos intrínsecos del ítem y no de su historial de valoraciones, el sistema puede generar recomendaciones desde el primer contacto con el usuario [2].

3. Motor de Recomendación Similitud del Coseno

El sistema de recomendación de Amazon demostró que es computacionalmente más eficiente calcular similitudes entre ítems que entre usuarios, especialmente en catálogos de millones de productos. En lugar de encontrar usuarios similares al activo, Amazon pre calcula para cada producto cuáles son sus ítems más parecidos, reduciendo la complejidad en tiempo de respuesta de $O(MN)$ a $O(N)$ [3].

La métrica central de este enfoque es la Similitud del Coseno, consolidada como el estándar de facto para sistemas de recomendación basados en contenido e ítem-ítem. A diferencia de la distancia euclidiana, mide el ángulo entre dos vectores en lugar de su separación absoluta, siendo invariante a la magnitud, con un rango $[-1, 1]$ donde 1 indica máxima similitud:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Esta propiedad es crucial en catálogos extensos: dos películas de géneros similares tendrán vectores TF-IDF apuntando en la misma dirección aunque sus magnitudes difieren por ejemplo, una sinopsis corta frente a una larga [3]. Las recomendaciones resultantes son coherentes con los atributos intrínsecos de los ítems, sin depender de patrones de comportamiento colectivo [3].

4. Factorización de Matrices SGD y Factores Latentes

La Netflix Prize Competition es el hito fundacional del filtrado colaborativo moderno. El equipo ganador BellKor's Pragmatic Chaos, liderado por Yehuda Koren, demostró que los modelos de Factorización de Matrices superan a las técnicas clásicas de vecindad más cercana, permitiendo incorporar retroalimentación implícita, efectos temporales y niveles de confianza [4].

Koren, Bell & Volinsky proponen descomponer la matriz de interacciones usuario-ítem en dos matrices de menor dimensión: una de factores de usuario (P) y una de factores de ítem (Q) [4]. Cada dimensión latente captura una característica oculta del gusto como la preferencia por películas de acción o dramas de época sin que sea necesario definirla explícitamente [4]. La predicción de valoración se calcula como:

$$\hat{r}_{ui} = b + b_u + b_i + p_u^T \cdot q_i$$

Donde b es la media global, b_u el sesgo del usuario y b_i el sesgo del ítem [4]. El algoritmo SGD actualiza los parámetros tras cada ejemplo de entrenamiento, estimando $\hat{r}_{ui}$ calculando el error $e_{ui} = r_{ui} - \hat{r}_{ui}$ y ajustando P y Q en la dirección que reduce dicho error, controlado por la tasa de aprendizaje α y el término de regularización β [4].

En este proyecto se diseñó un entorno de usuarios sintéticos para ejecutar SGD, dado que el dataset original no contenía valoraciones históricas para entrenar el filtro colaborativo problema de escasez de datos documentado precisamente por el equipo de Netflix [4].

5. Evaluación Sistémica Métricas de Error y Ranking

Gunawardana & Shani establecen el marco de referencia para la evaluación de sistemas de recomendación, distinguiendo entre métricas de exactitud predictiva y métricas de ranking [5]. Las métricas RMSE y MAE miden la distancia entre la valoración predicha y la real: el RMSE penaliza más los errores grandes y constituyó el estándar oficial de la Netflix Prize Competition para comparar modelos [4][5].

La métrica Precision@K representa la fracción de ítems relevantes entre los K primeros recomendados, siendo la más directamente interpretable: de los 10 primeros títulos sugeridos, ¿cuántos son realmente de su gusto? [5]. Complementariamente, el Recall@K mide qué proporción de los ítems relevantes totales aparecen en las K recomendaciones; el balance Precision-Recall define la utilidad global del sistema [5].

Estos criterios son consistentes con los marcos de evaluación utilizados por MovieLens (*offline evaluation* sobre particiones de *train/test*) y por Amazon (evaluación A/B en producción), lo que valida su aplicabilidad tanto en contextos académicos como industriales [1][3][5].

METODOLOGÍA

Para realizar el siguiente proyecto nos basamos en tres modelos y los separamos en 6 notebooks

1. Exploración de Datos

Paso 1: Carga y Consolidación de la Base de Datos

El objetivo inicial es unificar las fuentes de información dispersas para obtener una visión integral de cada título cinematográfico.

Ingesta de datos: Se realiza la lectura de los archivos fuente de Kaggle que contienen los metadatos y los créditos de las películas.

Unión técnica (Merge): Se vinculan ambas tablas mediante el identificador único de cada película. Esto genera un dataset consolidado que permite cruzar información como el presupuesto con los nombres de los actores y directores.

Paso 2: Auditoría de Estructura y Calidad de Datos

Antes de proceder con cualquier cálculo, se evalúa la "salud" del dataset para identificar posibles puntos de falla en el modelo.

Clasificación de variables: Se identifican qué columnas contienen datos numéricos y cuáles contienen texto o estructuras complejas.

Detección de vacíos: Se cuantifican los valores nulos para entender el impacto de los datos faltantes. Se documenta que columnas como la fecha de estreno y la duración presentan vacíos que deben ser corregidos en la etapa posterior.

Paso 3: Caracterización de Metadatos y Géneros

En este paso se analiza la información cualitativa que servirá de base para las recomendaciones por contenido.

Análisis de géneros: Se procesan las categorías cinematográficas para determinar la inclinación del catálogo (por ejemplo, predominancia de dramas o comedias).

Perfil de producción: Se identifican los actores y directores con mayor presencia, información que será fundamental para construir los perfiles de similitud entre películas.

Paso 4: Análisis Estadístico y Relacional

Se estudian las propiedades matemáticas de los datos para configurar los algoritmos de optimización.

Distribución de valoraciones: Se analiza cómo se reparten los ratings de los usuarios para verificar si siguen una distribución normal, lo cual es clave para la futura simulación de usuarios sintéticos.

Estudio de correlaciones: Se busca determinar si existe una relación directa entre variables numéricas, como el éxito comercial frente al presupuesto invertido, mediante el uso de mapas de calor estadísticos.

2. Preprocesamiento

Paso 1: Extracción y Limpieza de Estructuras Complejas

El objetivo es convertir la información almacenada en formatos difíciles de procesar (como listas de texto o JSON) en datos estructurados y limpios.

Parseo de metadatos: Se extraen de forma aislada los nombres de los géneros, las palabras clave, los actores principales y el director del equipo técnico.

Normalización de identificadores: Se eliminan los espacios en blanco de los nombres propios (ej. de "James Cameron" a "JamesCameron") para asegurar que el sistema trate cada nombre como una entidad única e inequívoca.

Paso 2: Generación del Perfil de Contenido (Tag Soup)

Se crea una representación textual densa para cada película que servirá de base para el filtrado por contenido.

Consolidación de etiquetas: Se fusionan los metadatos procesados (géneros, actores, director) con el resumen de la trama en una sola columna descriptiva.

Preparación para vectorización: Este bloque de texto consolidado es el que permitirá medir matemáticamente qué tan parecida es una película a otra en la siguiente fase.

Paso 3: Normalización de Variables Numéricas y Limpieza Final

Se ajustan los datos cuantitativos para que tengan escalas comparables y se eliminan registros incompletos.

Tratamiento de nulos: Se remueven definitivamente las películas con datos faltantes en fecha de estreno o duración, garantizando la continuidad del flujo de datos.

Escalamiento logarítmico: Se aplica una transformación a variables con gran varianza, como el presupuesto y los ingresos, para reducir el impacto de los valores extremos en los cálculos estadísticos.

Paso 4: Construcción de la Arquitectura de Usuarios Sintéticos

Dado que el dataset original no posee interacciones individuales, se simula una base de usuarios para habilitar el filtrado colaborativo.

Simulación de ratings: Se generan perfiles de usuarios ficticios y se les asignan calificaciones basadas en la tendencia de votos reales de la película más un factor de variabilidad aleatoria.

Declaración de limitación: Se establece este proceso como una simulación matemática necesaria para probar los algoritmos de optimización ante la falta de usuarios reales.

Paso 5: Segmentación para Entrenamiento y Prueba (Split)

Se divide la información en dos conjuntos para validar que los modelos realmente aprendan y no solo memoricen los datos.

Distribución 80/20: Se reserva una porción de los ratings para el entrenamiento de los algoritmos y otra para la evaluación posterior del error.

Paso 6: Exportación y Almacenamiento de Artefactos

Se guardan los resultados finales en formatos de alta eficiencia para ser consumidos por los notebooks de modelado.

Persistencia de datos: Se generan archivos de matrices numéricas (NumPy) y tablas limpias (CSV) que contienen toda la inteligencia procesada, evitando la necesidad de repetir la limpieza en cada ejecución de los modelos.

3. Filtrado Basado en Contenido

Paso 1: Representación Vectorial de Textos

Para que el sistema pueda procesar el "Tag Soup" generado en la etapa de preprocesamiento, se requiere transformar las palabras en valores numéricos.

Algoritmo Aplicado: Se utiliza la técnica TF-IDF (Frecuencia de Término – Frecuencia Inversa de Documento).

Lógica de Pesado: Esta herramienta matemática asigna una mayor puntuación a las palabras que son únicas y descriptivas de una película (como nombres de superhéroes o subgéneros específicos) y reduce el peso de las palabras comunes que no aportan valor predictivo.

Paso 2: Normalización de Longitud

Dado que algunas películas tienen descripciones más extensas que otras, es necesario estandarizar los vectores para que la comparación sea equitativa.

Técnica Aplicada: Se emplea la Normalización L2 (o Norma Euclidiana).

Efecto: Este proceso ajusta la magnitud de todos los vectores de película a un valor unitario, asegurando que el sistema se enfoque exclusivamente en la temática y no en la cantidad de palabras del texto original.

Paso 3: Cálculo de la Proximidad Temática

Una vez que las películas están representadas como vectores en un espacio multidimensional, el sistema mide qué tan cerca están entre sí.

Herramienta de Medición: Se aplica la Similitud Coseno.

Interpretación: El algoritmo calcula el ángulo entre los vectores de dos películas; cuanto más pequeño es el ángulo (y más cercano a 1 es el coseno), mayor es la similitud de contenido entre ambos filmes.

Paso 4: Generación del Ranking de Similitud

Con la matriz de similitudes calculada para todo el catálogo, el sistema activa la lógica de recomendación.

Operación de Selección: Se emplea un algoritmo de Ordenamiento (Sorting) sobre los puntajes de similitud generados.

Extracción de Resultados: El sistema identifica la película de referencia, busca a sus "vecinos" más cercanos en el espacio vectorial y entrega una lista jerarquizada con los títulos que poseen el ADN cinematográfico más parecido.

Paso 5: Validación de Resultados

Se realiza una auditoría cualitativa para verificar que las recomendaciones tengan sentido lógico para un usuario humano.

Prueba de Coherencia: Se ingresan títulos conocidos (ej. películas de ciencia ficción) y se comprueba que el algoritmo devuelve títulos del mismo género o con el mismo reparto y director, validando así la efectividad del procesamiento de lenguaje natural aplicado.

4. Matrix Factorization (MF) con SGD

Paso 1: Representación de Factores Latentes

El procedimiento inicia descomponiendo la matriz de interacciones en dos estructuras de menor dimensión que captaron características ocultas o "latentes".

Componente de Usuario: Se utiliza una matriz de perfiles que describe las preferencias individuales.

Componente de Ítem: Se utiliza una matriz de atributos que describe las propiedades de las películas.

Operación de Predicción: Para estimar un rating, el sistema aplica la operación de Producto Punto (Dot Product) entre los vectores correspondientes de ambas matrices.

Paso 2: Definición del Criterio de Error y Control de Ajuste

Para que el sistema aprenda, se define un objetivo matemático que cuantifica qué tan precisas son las predicciones.

Función de Error: Se utiliza la fórmula del Error Cuadrático Medio (MSE) para medir la diferencia entre los ratings reales y los predichos.

Control de Sobreajuste: Se aplica la técnica de Regularización L2 (Penalización de Ridge). Esta se encarga de que los valores internos del modelo no crezcan desproporcionadamente, garantizando que el sistema pueda generalizar sus recomendaciones a películas que el usuario no ha visto.

Paso 3: Optimización mediante Descenso de Gradiente Estocástico (SGD)

Este es el proceso central de entrenamiento, donde el modelo ajusta sus parámetros de forma iterativa.

Cálculo de Gradientes: El algoritmo calcula la Derivada Parcial de la función de error respecto a cada peso del modelo. El gradiente indica hacia dónde debe "moverse" el valor para reducir el error.

Actualización Iterativa: Se emplea el Descenso de Gradiente Estocástico (SGD), el cual procesa cada interacción una por una. En cada paso, se aplica una Tasa de Aprendizaje (Learning Rate) para controlar la velocidad y estabilidad del ajuste de los factores latentes.

Paso 4: Validación de Convergencia

El modelo se somete a múltiples ciclos de aprendizaje denominados Épocas.

Monitoreo: En cada época, se calcula el RMSE (Raíz del Error Cuadrático Medio) tanto en los datos conocidos como en los datos de prueba ocultos.

Gráfica de Convergencia: Se genera un reporte visual que demuestra cómo el error disminuye con el tiempo, lo cual sirve como evidencia técnica de que la optimización por SGD fue exitosa.

Paso 5: Generación de Recomendaciones

Una vez que el modelo ha convergido, se procede a la fase de entrega de resultados.

Cómputo Total: El sistema realiza la multiplicación de las matrices optimizadas para generar una tabla completa de predicciones para cada usuario.

Ranking y Filtrado: Se aplican algoritmos de Ordenamiento para seleccionar las películas con mayor puntaje estimado, excluyendo aquellas que el usuario ya ha consumido previamente en el set de entrenamiento.

5. KNN Colaborativo (Memory-Based)

Paso 1: Representación de la Matriz de Interacción

A diferencia de los modelos anteriores, este enfoque requiere que la información esté organizada en una estructura donde se crucen las identidades de los usuarios con las películas que han calificado.

Estructura de Datos: Se utiliza la matriz de Usuarios vs. Películas generadas en la fase de preprocesamiento, donde cada celda contiene el rating otorgado.

Tratamiento de Valores Faltantes: Se aplican técnicas de imputación (relleno con ceros) para representar las películas que un usuario aún no ha visto, permitiendo que los algoritmos de distancia operen sobre una matriz completa.

Paso 2: Cálculo de Cercanía entre Ítems

El sistema busca identificar qué películas son "vecinas" basándose exclusivamente en que han sido calificadas de manera similar por los mismos usuarios.

Métrica de Distancia: Se emplea la Similitud Coseno aplicada sobre los vectores de ratings.

Lógica de vecindad: Si dos películas reciben puntuaciones idénticas de la mayoría de los usuarios, su distancia matemática será mínima, indicando que son altamente parecidas para la comunidad.

Paso 3: Aplicación del Algoritmo K-Nearest Neighbors (KNN)

Una vez calculadas las distancias, el sistema utiliza el algoritmo de los K-Vecinos más Cercanos para generar las recomendaciones.

Búsqueda de Vecinos: Dado un título de película, el algoritmo identifica los k elementos con la menor distancia en la matriz de interacciones.

Predicción de Interés: El sistema estima que si al usuario le gustó la película "A", le gustarán sus vecinos más próximos, ya que otros usuarios con perfiles similares han validado esa relación mediante sus votos.

Paso 4: Generación de Listas de Recomendación

Se establece un proceso de ranking para entregar los resultados finales al usuario.

Filtrado de Candidatos: Se excluyen del resultado la película de consulta y aquellas que el usuario ya conoce.

Ordenamiento por Puntuación: Se presentan las películas vecinas ordenadas por su nivel de cercanía matemática, garantizando que los primeros resultados sean los que tengan mayor respaldo estadístico en la comunidad de usuarios.

Paso 5: Comparación con Modelos Anteriores

En este paso final del notebook, se realiza una verificación de consistencia frente a los resultados de los modelos de Contenido y Factorización.

Análisis de Resultados: Se documenta cómo el KNN, al ser un modelo "basado en memoria", ofrece recomendaciones muy directas pero depende críticamente de la cantidad de ratings disponibles en la matriz.

6. Evaluación y Comparación de Modelos

Paso 1: Consolidación de Predicciones y Set de Prueba

Para asegurar una competencia justa, todos los modelos deben ser evaluados bajo las mismas condiciones y con los mismos datos que no "vieron" durante el entrenamiento.

Recuperación de Datos: Se cargan los ratings reales del Conjunto de Prueba (Test Set) que fue reservado en el Notebook 02.

Generación de Inferencias: Se ejecutan los tres modelos para predecir las valoraciones sobre este conjunto específico de películas y usuarios.

Paso 2: Cálculo de Métricas de Error (Precisión Numérica)

Se evalúa qué tan cerca estuvieron las predicciones matemáticas de los valores reales asignados en la simulación.

RMSE (Raíz del Error Cuadrático Medio): Se utiliza esta métrica para penalizar los errores grandes en la predicción. Es la métrica estándar en sistemas de recomendación para medir la desviación promedio.

MAE (Error Absoluto Medio): Se emplea para obtener una visión más lineal del error promedio, sin amplificar tanto las diferencias extremas.

Paso 3: Cálculo de Métricas de Ranking (Calidad de la Recomendación)

A diferencia del error numérico, aquí se mide si el sistema es capaz de poner las películas "buenas" al principio de la lista.

Precision@K: Mide qué porcentaje de las películas recomendadas en el "Top K" fueron realmente del agrado del usuario.

Recall@K: Mide cuántas de las películas que le gustaban al usuario fueron capturadas dentro de las recomendaciones entregadas.

Paso 4: Visualización Comparativa de Resultados

Se procesan los resultados obtenidos para convertirlos en evidencia visual para el informe final.

Gráficos de Barras: Se comparan los modelos lado a lado bajo las métricas de error y precisión.

Curvas de Aprendizaje: Se analiza el comportamiento de convergencia del modelo SGD frente a los otros enfoques para determinar la estabilidad del algoritmo de optimización.

Experimentación y Resultados

En esta sección se detalla la implementación técnica y los hallazgos cuantitativos del modelo K-Nearest Neighbors (KNN) desarrollado de forma manual.

5.1 Configuración del Experimento

Se utilizaron las matrices procesadas ratings_train.npy y ratings_test.npy. La experimentación se centró en la optimización del hiperparámetro K (número de vecinos), evaluando valores de K que pertenecen {5, 10, 20, 50}. El cálculo de similitud se realizó mediante la Similitud de Coseno, implementada a través de operaciones vectorizadas en NumPy para maximizar la eficiencia numérica.

5.2 Resultados Cuantitativos

Los resultados obtenidos tras la ejecución del script de evaluación se resumen en la siguiente tabla.

Valor de K	RMSE (Error Cuadrático Medio)	MAE (Error Absoluto Medio)
5	1.1245	0.8542
10	1.0532	0.7921
20 (Óptimo)	1.0120	0.7510
50	1.0344	0.7745

Discusión

El análisis de los resultados permite extraer las siguientes observaciones desde la perspectiva de Métodos Numéricos y Optimización:

1. Estabilidad de la similitud: La implementación manual de la similitud de coseno demostró ser robusta. Al incluir un factor de estabilidad $\epsilon = 1 \times 10^{-9}$ en el denominador, se evitaron errores de división por cero en películas con pocos ratings, garantizando que el algoritmo no colapse ante datos dispersos (sparsity).
2. Comportamiento del Hiperparámetro K: Se observó que un K muy bajo genera "ruido" en la recomendación (overfitting a vecinos atípicos), mientras que un K muy alto suaviza demasiado las predicciones. El valor de $K=20$ resultó ser el punto de equilibrio donde el error fue mínimo.
3. Eficiencia Computacional: Al evitar frameworks como scikit-learn y utilizar el producto punto matricial ($A \cdot B^T$), se logró reducir el tiempo de cálculo de la matriz de similitud de minutos a segundos, aprovechando la optimización intrínseca de los arreglos de NumPy.

Conclusiones y trabajos futuros

- Se logró diseñar un sistema de recomendación funcional utilizando métodos directos de álgebra lineal, cumpliendo con la restricción de no usar frameworks de modelado automáticos.
- El uso de formatos binarios (.npy) fue crítico para la persistencia y velocidad de carga de los datos, demostrando que la estructura de los datos es tan importante como el algoritmo mismo.
- La validación mediante RMSE manual permitió confirmar que el modelo tiene una precisión aceptable para una base de datos de 5000 películas.

Trabajos Futuros:

- **Paralelización:** Implementar el cálculo de similitudes usando programación paralela o GPU (Cuda) para escalar el sistema a millones de usuarios.
- **Hibridación:** Integrar técnicas de Factorización de Matrices para resolver el problema de datos faltantes que el KNN por sí solo no puede cubrir.

FUENTES

1. **GroupLens Research.** (s.f.). *MovieLens Dataset*. Universidad de Minnesota. Disponible en: <https://grouplens.org/datasets/movielens/>
2. **Pandora Media.** (s.f.). *Pandora Music Genome Project*. Disponible en: <https://www.pandora.com/about/mgp>
3. **Linden, G., Smith, B., & York, J.** (2003). *Amazon.com Recommendations: Item-to-Item Collaborative Filtering*. IEEE Internet Computing, 7(1), 76-80. Disponible en: <https://ieeexplore.ieee.org/document/1167344>
4. **Koren, Y., Bell, R., & Volinsky, C.** (2009). *Matrix Factorization Techniques for Recommender Systems*. IEEE Computer Society. Disponible en: [https://datajobs.com/data-science-repo/Recommender-Systems-\[Netflix\].pdf](https://datajobs.com/data-science-repo/Recommender-Systems-[Netflix].pdf)
5. **Gunawardana, A. & Shani, G.** (2015). *Evaluating Recommender Systems*. En: Recommender Systems Handbook. Springer. Disponible en: https://link.springer.com/chapter/10.1007/978-1-4899-7637-6_8